FlaskGuard

Custom Web Application Firewall

Technical Report – Part 4

File Upload Scanner, HTTPS/TLS Encryption,
& Secrets Hardening

Repository:	`github.com/echoenvoy/FlaskGuard`
Report Part:	4 of 5
Responsable:	Mohamed Ben Mouh
Key Files:	`file_scanner.py`
Date:	June 2026

Contents

1 Introduction **2**
1.1 The File Upload Attack Surface . . . 2
1.2 Scope . . . 2

2 File Upload Scanner – `file_scanner.py` **2**
2.1 Architecture Overview . . . 2
2.2 Layer 1: Filename Sanitisation . . . 2
2.2.1 Path Traversal in Filename . . . 3
2.3 Layer 2: Extension Validation . . . 3
2.4 Layer 3: Magic Byte Verification . . . 3
2.5 Layer 4: Text Content Scanning . . . 3
2.6 Layer 5: SVG-Specific Injection Detection . . . 3
2.7 Layer 6: Archive Bomb Detection . . . 3
2.8 Layer 7: Office Document Macro Detection . . . 4
2.9 Double-Extension Detection . . . 4
2.10 Scanner Return Value and Integration . . . 4

3 HTTPS/TLS Encryption **4**
3.1 Why TLS Matters for a WAF . . . 4
3.2 TLS Termination Architecture . . . 4
3.3 Nginx TLS Configuration and IP Forwarding . . . 4
3.4 Backend TLS Verification . . . 5

4 Conclusion **5**

Introduction

The File Upload Attack Surface

File upload endpoints are among the most dangerous attack surfaces in any web application. A malicious file upload can lead to:

- **Remote Code Execution (RCE):** A PHP webshell uploaded to a web-accessible directory and executed by the web server.
- **Stored XSS:** An SVG file containing scripts served with an image content type.
- **Denial of Service:** A zip bomb – a small file that expands to gigabytes when decompressed – exhausting disk space or CPU.
- **Malware delivery:** Office documents with embedded macros that execute on the victim's system when opened.
- **Path traversal via filename:** A filename like `../../etc/cron.d/backdoor` placed in a directory where the server stores uploaded files.

A WAF operating at the HTTP level has the opportunity to inspect file uploads *before* they reach the backend application, applying a battery of checks that the application may not implement.

Scope

- **file_scanner.py:** Seven-layer file upload inspection pipeline
- **HTTPS/TLS:** TLS termination architecture and configuration recommendations

File Upload Scanner – `file_scanner.py`

Architecture Overview

The file scanner is invoked by the `before_request` hook when the incoming request has a `Content-Type` of `multipart/form-data`, indicating a file upload. The scanner applies its checks in a defined sequence:

Layer	Check
1	Filename sanitisation and path traversal in filename
2	Extension allowlist/blocklist
3	Magic byte verification
4	Text content scanning (regex patterns)
5	SVG-specific script injection detection
6	Archive bomb detection
7	Office document macro detection

Checks run in order of cost: inexpensive checks (filename, extension) run first. If any check fails, the file is rejected immediately without running the remaining checks.

Layer 1: Filename Sanitisation

The filename submitted by the client cannot be trusted. The scanner applies two checks:

Path Traversal in Filename

The system explicitly rejects null bytes (often used in language-level exploits), path traversal sequences like `../` or `..\`, and absolute paths. Following these custom checks, FlaskGuard utilizes Werkzeug's `secure_filename()` utility to strip any remaining dangerous characters or path components, ensuring the filename is safe for the filesystem.

Layer 2: Extension Validation

Extensions are validated against a blocklist of dangerous types (such as web executables like `.php`, `.asp`, server-side scripts like `.py`, `.sh`, and Windows executables like `.exe`, `.bat`).

Layer 3: Magic Byte Verification

File type is determined not from the filename extension or the content-type header (both attacker-controlled) but from the file's **magic bytes** – the first few bytes of the file content that identify its true format via the `python-magic` library.

- **Dangerous Mime Types:** It blocks clearly dangerous formats (like PHP or executable binaries) immediately, regardless of what extension the user claimed.
- **Mismatch Detection:** It compares the detected true type against an expected map for common extensions (like ensuring a `.jpg` doesn't return PDF magic bytes), rejecting mismatches outright.

Layer 4: Text Content Scanning

For files that pass the extension and magic byte checks, text-based content scanning attempts to decode the file data as UTF-8. If successful, it applies the same regex patterns used in the request body scanner (checking for SQL injection, XSS, command injection, etc.) directly against the file's text contents. Binary files simply bypass this step.

Layer 5: SVG-Specific Injection Detection

SVG files require special handling because they are XML-based and may legally contain JavaScript. A browser serving an SVG with an image content type will execute embedded scripts, enabling stored XSS. The scanner parses the XML structure and actively blocks the file if it finds embedded `<script>` tags, dangerous `on*` event handler attributes, or `href` attributes pointing to a `javascript:` URI.

Layer 6: Archive Bomb Detection

A zip bomb is a specially crafted archive file that appears small but expands to a massive size when decompressed, potentially causing disk or CPU exhaustion. FlaskGuard mitigates this by checking the archive's metadata. It calculates the total uncompressed size and the compression ratio, immediately rejecting files that exceed safe limits (e.g., uncompressed sizes over 50 MB or a compression ratio greater than 100:1).

Layer 7: Office Document Macro Detection

Microsoft Office documents can contain VBA macros that execute code on the victim's machine. Because Office Open XML files are essentially ZIP archives, FlaskGuard unzips them in memory and scans for known macro indicators (such as `vbaProject.bin` or macro sheets) and rejects the document if hidden automation code is found.

Double-Extension Detection

A filename like `malware.php.jpg` attempts to fool basic filters by placing an innocent extension last. The scanner checks if a filename contains multiple dots; if the inner extension matches a blocked executable type (like `.php`), it flags the upload as a double-extension trick and blocks it.

Scanner Return Value and Integration

The scanner runs within a `before_request` hook for incoming `POST` requests. It safely reads the incoming file stream, runs the validation pipeline, and ensures the stream position is reset (seeking back to zero) so proxy forwarding isn't broken. If any layer returns a rejection reason, the event is logged as a `MALICIOUS_UPLOAD` and the endpoint returns a JSON response with an HTTP 403 status code.

HTTPS/TLS Encryption

Why TLS Matters for a WAF

A WAF operating without TLS faces two critical vulnerabilities:

- **Credential exposure:** Administrative credentials sent via basic authentication are merely base64 encoded and travel in cleartext over the network.
- **Request tampering:** Man-in-the-middle attackers can alter requests in transit, potentially bypassing WAF inspection.

TLS Termination Architecture

Because Python-based Flask applications are not optimized to handle encryption overhead at scale, TLS termination should be offloaded to a dedicated reverse proxy layer sitting in front of the application. The traffic flows as follows: `Client (HTTPS)` → `Nginx/Caddy/Traefik (TLS Termination)` → `FlaskGuard (Internal HTTP)` → `Backend (Internal HTTP)`. This separation of concerns ensures the proxy handles cipher negotiation and certificates, while FlaskGuard focuses purely on security inspection.

Nginx TLS Configuration and IP Forwarding

To deploy this, Nginx is configured to listen on port 443 with SSL certificates, modern TLS protocols (1.2 and 1.3), and strict security headers (such as HSTS, X-Frame-Options, and X-Content-Type-Options). It uses a `proxy_pass` directive to forward requests internally to FlaskGuard.

Because the proxy sits in the middle, Flask's native `request.remote_addr` will perceive the proxy's local IP instead of the actual client. To fix this, FlaskGuard applies Werkzeug's `ProxyFix` middleware to safely trust the forwarded headers (like `X-Forwarded-For`).

Backend TLS Verification

Configuring the proxy subsystem with `verify_tls: true` ensures that when FlaskGuard forwards inspected requests further downstream to the actual backend application, it enforces certificate validation, preventing man-in-the-middle interception on the internal network.

Conclusion

Part 4 of the FlaskGuard technical report establishes robust defenses against malicious files and unencrypted communication. The multi-layered file upload scanner efficiently blocks dangers—ranging from path traversal and double-extension tricks to archive bombs and VBA macros—by running cheaper, filename-based checks before moving on to resource-heavy content inspection.

Furthermore, offloading TLS termination to a reverse proxy preserves data integrity and protects credentials without burdening the Flask application. Using proper header forwarding configurations like `ProxyFix` ensures that rate-limiting and IP restrictions remain accurate, elevating FlaskGuard to an enterprise-grade web application firewall.